Reinforcement Learning Practicals

Ques1.Dynamic Programming Policy Evaluation algorithm.

```
import numpy as np

class GridWorld:
    def __init__(self, size=4):
        self.size = size
        self.n_states = size * size
        self.n_actions = 4  # up, down, left, right
        self.terminal_states = [0, self.n_states - 1]

    def step(self, state, action):
        if state in self.terminal_states:
            return state, 0

        row, col = divmod(state, self.size)

        if action == 0: row = max(row - 1, 0)
        elif action == 1: row = min(row + 1, self.size - 1)
        elif action == 2: col = max(col - 1, 0)
        elif action == 3: col = min(col + 1, self.size - 1)

        next_state = row * self.size + col
        reward = -1
        return next_state, reward


def policy_evaluation(env, policy, gamma=1.0, theta=1e-5):
    V = np.zeros(env.n_states)

    while True:
        delta = 0
        for s in range(env.n_states):
            v = 0
            for a, prob in enumerate(policy[s]):
                next_s, r = env.step(s, a)
                v += prob * (r + gamma * V[next_s])
            delta = max(delta, abs(V[s] - v))
            V[s] = v
        if delta < theta:
            break
    return V
```

```
env = GridWorld()
policy = np.ones((env.n_states, env.n_actions)) / env.n_actions
V = policy_evaluation(env, policy)
print(V.reshape(4,4))
```

```
[[  0.         -13.99993529 -19.99990698 -21.99989761]
 [-13.99993529 -17.9999206  -19.99991379 -19.99991477]
 [-19.99990698 -19.99991379 -17.99992725 -13.99994569]
 [-21.99989761 -19.99991477 -13.99994569   0.        ]]
```

Ques2. Dynamic Programming Policy Iteration algorithm.

```
def policy_iteration(env):
    policy = np.ones((env.n_states, env.n_actions)) / env.n_actions

    while True:
        V = policy_evaluation(env, policy)
        stable = True

        for s in range(env.n_states):
            old = np.argmax(policy[s])
            values = []

            for a in range(env.n_actions):
                next_s, r = env.step(s, a)
                values.append(r + V[next_s])

            best = np.argmax(values)
            policy[s] = np.eye(env.n_actions)[best]

            if old != best:
                stable = False

        if stable:
            break

    return policy, V

policy_pi, V_pi = policy_iteration(env)
print(policy_pi)
```

```
[[1. 0. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 1. 0.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [1. 0. 0. 0.]
 [1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 1. 0. 0.]
 [1. 0. 0. 0.]
 [0. 0. 0. 1.]
 [0. 0. 0. 1.]
 [1. 0. 0. 0.]]
```

Ques3. Dynamic Programming Value Iteration algorithm.

```
def value_iteration(env):
    V = np.zeros(env.n_states)

    while True:
        delta = 0
        for s in range(env.n_states):
            values = []
            for a in range(env.n_actions):
                next_s, r = env.step(s, a)
                values.append(r + V[next_s])

            max_v = max(values)
            delta = max(delta, abs(V[s] - max_v))
            V[s] = max_v

        if delta < 1e-5:
            break

    return V

V_vi = value_iteration(env)
print(V_vi.reshape(4,4))
```

```
[[ 0. -1. -2. -3.]
 [-1. -2. -3. -2.]
 [-2. -3. -2. -1.]
 [-3. -2. -1.  0.]]
```

Ques4. Monte Carlo Prediction.

```
env_bj = gym.make("Blackjack-v1")

def mc_prediction(env, episodes=5000):
    V = defaultdict(float)
    returns = defaultdict(list)

    def policy(s): return 0 if s[0] >= 20 else 1

    for _ in range(episodes):
        episode = []
        s, _ = env.reset()

        done = False
        while not done:
            a = policy(s)
            next_s, r, done, _, _ = env.step(a)
            episode.append((s, r))
            s = next_s

        G = 0
        for s, r in reversed(episode):
            G += r
            returns[s].append(G)
            V[s] = np.mean(returns[s])

    return V

V_mc = mc_prediction(env_bj)
print("MC done")

# Show 10 learned state values
for i, (k, v) in enumerate(V_mc.items()):
    print(k, ":", round(v, 3))
    if i == 10:
        Break
```

```
MC done
(13, 10, 0) : -0.545
(15, 4, 0) : -0.396
(18, 4, 1) : -1.0
(15, 4, 1) : 0.143
(19, 10, 0) : -0.677
(18, 10, 0) : -0.694
(14, 2, 0) : -0.4
(17, 4, 0) : -0.642
(12, 10, 0) : -0.531
(21, 9, 0) : 0.953
(18, 9, 0) : -0.56
```

Ques5. Off-Policy Monte Carlo Control with Importance Sampling

```
def off_policy_mc(env, episodes=5000):
    Q = defaultdict(lambda: np.zeros(env.action_space.n))
    C = defaultdict(lambda: np.zeros(env.action_space.n))

    for _ in range(episodes):
        episode = []
        s, _ = env.reset()

        done = False
        while not done:
            a = env.action_space.sample()
            next_s, r, done, _, _ = env.step(a)
            episode.append((s, a, r))
            s = next_s

        G, W = 0, 1
        for s, a, r in reversed(episode):
            G += r
            C[s][a] += W
            Q[s][a] += (W / C[s][a]) * (G - Q[s][a])
            if a != np.argmax(Q[s]):
                break
            W *= env.action_space.n

    return Q

Q_mc = off_policy_mc(env_bj)
# Show some Q-values
for i, (k, v) in enumerate(Q_mc.items()):
    print(k, ":", v)
```

```
    if i == 5:
        Break
(15, 9, 0) : [-0.73333333  0.        ]
(12, 10, 0) : [-0.55555556 -0.36585366]
(12, 4, 0) : [-0.07692308  0.47368421]
(21, 5, 0) : [ 0.93333333 -1.        ]
(12, 5, 0) : [-0.30434783 -0.29411765]
(17, 10, 0) : [-0.475 -0.5  ]
```

Ques6. SARSA On policy TD learning algorithm.

```
def sarsa(env, episodes=5000):
    Q = np.zeros((env.n_states, env.n_actions))

    for _ in range(episodes):
        s = np.random.randint(env.n_states)
        a = np.random.choice(env.n_actions)

        for _ in range(100):
            next_s, r = env.step(s, a)
            next_a = np.argmax(Q[next_s])
            Q[s][a] += 0.1 * (r + 0.99 * Q[next_s][next_a] - Q[s][a])
            s, a = next_s, next_a

    return Q

Q_sarsa = sarsa(env)
print("SARSA Q-table:")
print(Q_sarsa.reshape(4,4,4))
```

```
SARSA Q-table:
[[[ 0.          0.          0.          0.        ]
  [-1.98971719 -2.96970753 -1.         -2.96969376]
  [-2.96905321 -3.94008114 -1.99       -3.93868405]
  [-3.93034358 -2.9701     -2.9701     -3.93972501]]

 [[-1.         -2.96920026 -1.98991055 -2.96913772]
  [-1.99       -3.93888299 -1.99       -3.93740483]
  [-2.97009974 -2.97009972 -2.97009972 -2.97009972]
  [-3.93994498 -1.99       -3.93893372 -2.96988235]]

 [[-1.99       -3.93716816 -2.9693586  -3.94021901]
  [-2.97009993 -2.97009993 -2.97009992 -2.97009992]
  [-3.93891622 -1.99       -3.93901575 -1.99      ]
  [-2.96984366 -1.         -2.96926428 -1.98961431]]

 [[-2.9701     -3.94002275 -3.93959891 -2.9701    ]
  [-3.9400956  -2.96977095 -3.94019078 -1.99      ]
  [-2.96818347 -1.98978626 -2.96916151 -1.        ]
  [ 0.          0.          0.          0.        ]]]
```

Ques7. Q-learning OFF policy TD learning algorithm.

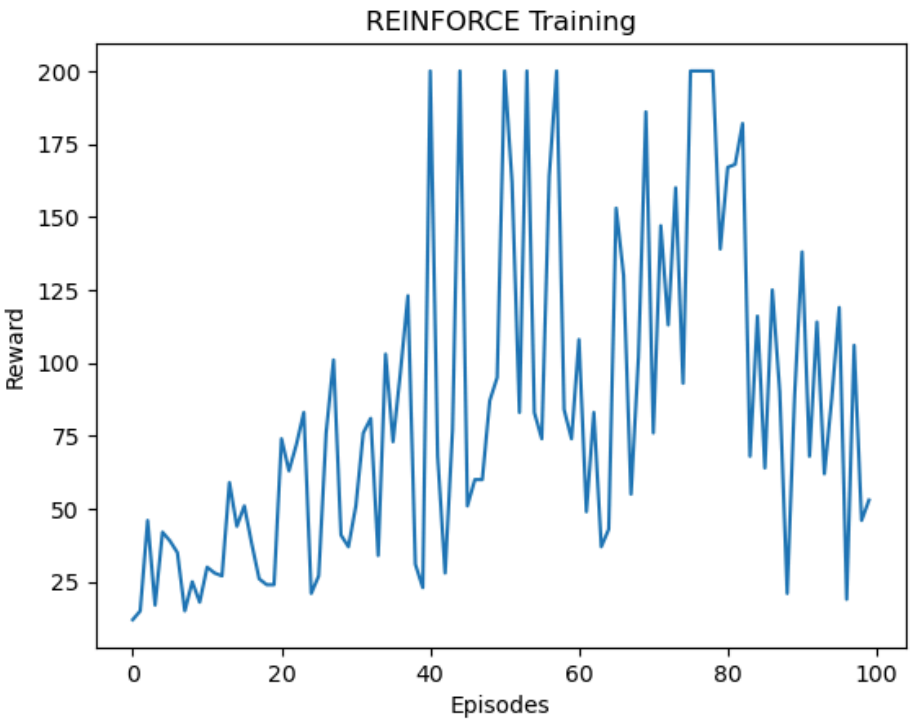
```
def q_learning(env, episodes=5000):
    Q = np.zeros((env.n_states, env.n_actions))

    for _ in range(episodes):
        s = np.random.randint(env.n_states)

        for _ in range(100):
            a = np.argmax(Q[s])
            next_s, r = env.step(s, a)
            Q[s][a] += 0.1 * (r + 0.99 * np.max(Q[next_s]) - Q[s][a])
            s = next_s

    return Q

Q_q = q_learning(env)
print("Q-Learning Q-table:")
print(Q_q.reshape(4,4,4))
```

```
Q-Learning Q-table:
[[[ 0.          0.          0.          0.        ]
  [-1.09451647 -1.15518465 -1.         -1.08115828]
  [-2.0791324  -2.05237512 -1.99       -2.1008441 ]
  [-3.05394637 -2.97009993 -2.97009993 -3.05394637]]

 [[-1.         -1.04704516 -1.09451647 -1.01211424]
  [-1.99       -2.00514159 -1.99       -2.13160989]
  [-2.96938613 -2.96939808 -2.96937002 -2.96939962]
  [-2.09198097 -1.99       -2.12685078 -2.0791324 ]]

 [[-1.99       -2.05953526 -2.0791324  -2.05811548]
  [-2.96956768 -2.96955618 -2.96956225 -2.96958225]
  [-1.99662387 -1.99       -2.15894389 -1.99      ]
  [-1.10558113 -1.         -1.05547326 -1.09451647]]

 [[-2.97009983 -3.05394637 -3.05394637 -2.97009983]
  [-2.00362885 -2.0791324  -2.13220569 -1.99      ]
  [-1.00496219 -1.09451647 -1.09424455 -1.        ]
  [ 0.          0.          0.          0.        ]]]
```

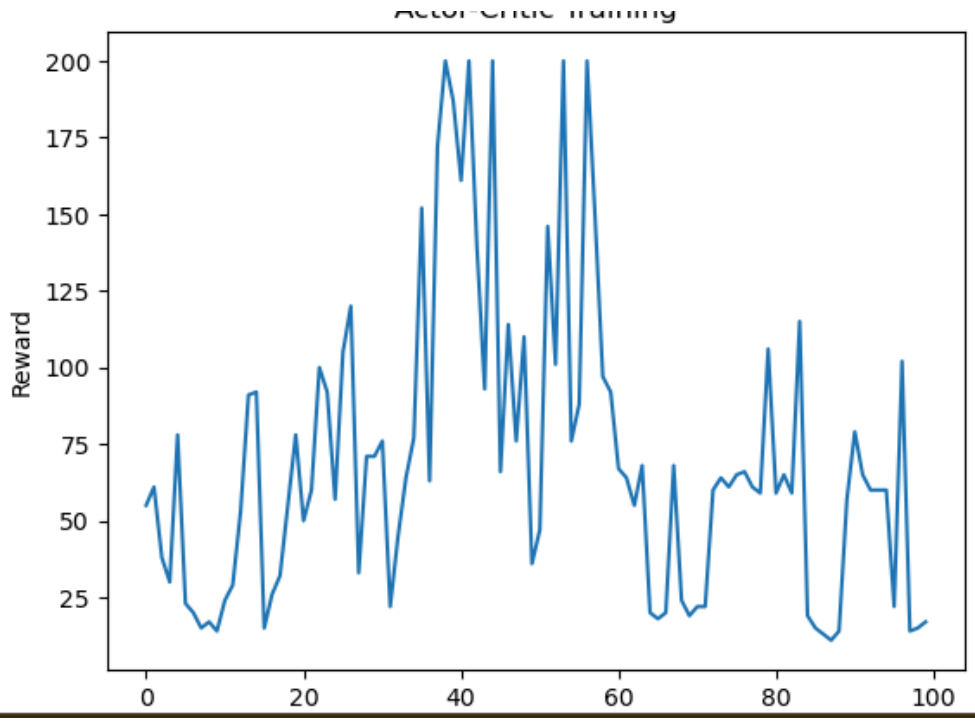
Ques8. Policy Gradient REINFORCE algorithm .

```
env_cp = gym.make("CartPole-v1")

model = tf.keras.Sequential([
    layers.Dense(24, activation='relu'),
    layers.Dense(2, activation='softmax')
])

optimizer = tf.keras.optimizers.Adam(0.01)
rewards_history = []

def reinforce(env):
    for episode in range(100):  # reduced
        states, actions, rewards = [], [], []
        s, _ = env.reset()

        for step in range(200):  # limit steps
            probs = model(np.array([s]))[0].numpy()
            a = np.random.choice(2, p=probs)

            next_s, r, done, _, _ = env.step(a)

            states.append(s)
            actions.append(a)
```

```
        rewards.append(r)

        s = next_s
        if done:
            break

    rewards_history.append(sum(rewards))

    G = np.cumsum(rewards[::-1])[::-1]

    with tf.GradientTape() as tape:
        loss = 0
        for s, a, g in zip(states, actions, G):
            probs = model(np.array([s]))
            loss += -tf.math.log(probs[0][a]) * g

    grads = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(grads, model.trainable_variables))

reinforce(env_cp)

plt.plot(rewards_history)
plt.title("REINFORCE Training")
plt.xlabel("Episodes")
plt.ylabel("Reward")
plt.show()
```



Ques9. Policy Gradient Actor-Critic method algorithm.

```
inputs = layers.Input(shape=(4,))
x = layers.Dense(128, activation='relu')(inputs)
action = layers.Dense(2, activation='softmax')(x)
critic = layers.Dense(1)(x)

model_ac = tf.keras.Model(inputs=inputs, outputs=[action, critic])
optimizer = tf.keras.optimizers.Adam(0.01)

ac_rewards = []

def actor_critic(env):
    for episode in range(100):  # reduced
        s, _ = env.reset()
        total_reward = 0

        for step in range(200):  # limit steps
            with tf.GradientTape() as tape:
                probs, value = model_ac(np.array([s]))
                a = np.random.choice(2, p=probs[0].numpy())

                next_s, r, done, _, _ = env.step(a)
                total_reward += r

                _, next_value = model_ac(np.array([next_s]))

                td = r + 0.99 * next_value - value
                loss = -tf.math.log(probs[0][a]) * td + td**2

            grads = tape.gradient(loss, model_ac.trainable_variables)
            optimizer.apply_gradients(zip(grads, model_ac.trainable_variables))

            s = next_s
            if done:
                break

        ac_rewards.append(total_reward)

actor_critic(env_cp)
plt.plot(ac_rewards)
plt.title("Actor-Critic Training")
plt.xlabel("Episodes")
plt.ylabel("Reward")
```

```
plt.show()
```